

On This Page >



Connecting to Your Machines

Clan provides automatic networking with fallback mechanisms to reliably connect to your machines.

Option 1: Automatic Networking with Fallback (Recommended)

Clan's networking module automatically manages connections through various network technologies with intelligent fallback. When you run `clan ssh` or `clan machines update`, Clan tries each configured network by priority until one succeeds.

Basic Setup with Internet Service

For machines with public IPs or DNS names, use the `internet` service to configure direct SSH while keeping fallback options:

flake.nix

```
1 {
2   outputs = { self, clan-core, ... }:
3     let
4       clan = clan-core.lib.clan {
5         inventory.instances = {
6           # Direct SSH with fallback support
7           internet = {
8             roles.default.machines.server1 = {
9               settings.host = "server1.example.com";
10            };
11          };
12        };
13      };
14   }
```

```
11         roles.default.machines.server2 = {
12             settings.host = "192.168.1.100";
13         };
14     };
15
16     # Fallback: Secure connections via Tor
17     tor = {
18         roles.server.tags.nixos = { };
19     };
20 };
21 };
22 in
23 {
24     inherit (clan.config) nixosConfigurations;
25 };
26 }
```

Advanced Setup with Multiple Networks

flake.nix

```
1 {
2     outputs = { self, clan-core, ... }:
3         let
4             clan = clan-core.lib.clan {
5                 inventory.instances = {
6                     # Priority 1: Try direct connection first
7                     internet = {
8                         roles.default.machines.publicserver = {
9                             settings.host = "public.example.com";
10                         };
11                     };
12
13                     # Priority 2: VPN for internal machines
14                     zerotier = {
15                         roles.controller.machines."controller" = { };
16                         roles.peer.tags.nixos = { };
17                     };
18
19                     # Priority 3: Tor as universal fallback
20                     tor = {
21                         roles.server.tags.nixos = { };
22                     };
23                 };
24             };
25 }
```

```
24     };  
25     in  
26     {  
27         inherit (clan.config) nixosConfigurations;  
28     };  
29 }
```

How It Works

Clan automatically tries networks in order of priority:

1. Direct internet connections (if configured)
2. VPN networks (ZeroTier, Tailscale, etc.)
3. Tor hidden services
4. Any other configured networks

If one network fails, Clan automatically tries the next.

Useful Commands

```
1 # View all configured networks and their status  
2 clan network list  
3  
4 # Test connectivity through all networks  
5 clan network ping machine1  
6  
7 # Show complete network topology  
8 clan network overview
```

Option 2: Manual targetHost (Bypasses Fallback!)



Setting `targetHost` directly **disables all automatic networking and fallback**. Only use this if you need complete control and don't want Clan's intelligent connection management.

Using Inventory (For Static Addresses)

Use inventory-level `targetHost` when the address is **static** and doesn't depend on NixOS configuration:

flake.nix

```
1 {
2   outputs = { self, clan-core, ... }:
3     let
4       clan = clan-core.lib.clan {
5         inventory.machines.server = {
6           # WARNING: This bypasses all networking modules!
7           # Use for: Static IPs, DNS names, known hostnames
8           deploy.targetHost = "root@192.168.1.100";
9         };
10      };
11    in
12      {
13        inherit (clan.config) nixosConfigurations;
14      };
15 }
```

When to use inventory-level:

- Static IP addresses: `"root@192.168.1.100"`
- DNS names: `"user@server.example.com"`
- Any address that doesn't change based on machine configuration

Using NixOS Configuration (For Dynamic Addresses)

Use machine-level `targetHost` when you need to **interpolate values from the NixOS configuration**:

flake.nix

```

1 {
2   outputs = { self, clan-core, ... }:
3     let
4       clan = clan-core.lib.clan {
5         machines.server = { config, ... }: {
6           # WARNING: This also bypasses all networking modules!
7           # REQUIRED for: Addresses that depend on NixOS config
8           clan.core.networking.targetHost = "root@${config.networking.hos
9         };
10      };
11    in
12      {
13        inherit (clan.config) nixosConfigurations;
14      };
15 }

```

When to use machine-level (NixOS config):

- Using hostName from config: `"root@${config.networking.hostName}.local"`
- Building from multiple config values:
`"${config.users.users.deploy.name}@${config.networking.hostName}"`
- Any address that depends on evaluated NixOS configuration

Key Difference

Inventory-level (`deploy.targetHost`) is evaluated immediately and works with static strings.

Machine-level (`clan.core.networking.targetHost`) is evaluated after NixOS configuration and can access `config.*` values.

Quick Decision Guide

Scenario	Recommended Approach	Why
Public servers	<code>internet</code> service	Keeps fallback options
Mixed infrastructure	Multiple networks	Automatic failover
Machines behind NAT	ZeroTier/Tor	NAT traversal with fallback
Testing/debugging	Manual targetHost	Full control, no magic

Scenario	Recommended Approach	Why
Single static machine	Manual targetHost	Simple, no overhead

Command-Line Override

The `--target-host` flag bypasses ALL networking configuration:

```
1 # Emergency access - ignores all networking config
2 clan machines update server --target-host root@backup-ip.com
3
4 # Direct SSH - no fallback attempted
5 clan ssh laptop --target-host user@10.0.0.5
```

Use this for debugging or emergency access when automatic networking isn't working.

← Previous

Advanced Example

Next →
Zerotier VPN

Need help? Reach out to [our community](#).